

Labo 02 — Images, couches et registries

Théorie — comment une image est nommée, de quoi elle est faite, d'où elle vient, et pourquoi Podman refuse de deviner.

Objectifs

- Décomposer un nom d'image complet et savoir ce que Docker complète implicitement — et ce que Podman refuse de compléter.
- Comprendre pourquoi un **tag** est une étiquette mouvante et un **digest** une identité.
- Expliquer le modèle en **couches** et ce qu'il implique pour le disque et le réseau.
- Savoir inspecter une image sans la lancer.
- Situer Docker Hub, les registries privés et les images « officielles ».

1. Le nom complet d'une image

Vous écrivez `podman pull postgres:16-alpine`. Le moteur, lui, comprend :

```
docker.io / library / postgres : 16-alpine
└─registry┐ └─namespace┐ └─repo┐ └─tag┐
```

Partie	Rôle	Valeur par défaut
registry	Le serveur qui héberge l'image	<code>docker.io</code> (Docker Hub)
namespace	L'organisation ou l'utilisateur propriétaire	<code>library</code> (images officielles)
repository	Le nom de l'application	<i>obligatoire</i>
tag	La version	<code>latest</code>

Trois conséquences immédiates :

- `postgres` seul signifie `docker.io/library/postgres:latest`.
- Une image d'entreprise porte un nom complet : `registry.masociete.be/equipe-paiement/api-facturation:1.4.2`. Un **point** ou un **port** dans la première partie signale un registry et non un namespace.
- Les images du namespace `library` sont les **images officielles** : maintenues avec Docker, auditées, documentées (`postgres`, `nginx`, `node`, `eclipse-temurin`). Une image `bobdu59/postgres` n'a aucune de ces garanties.

PODMAN

Docker complète `postgres` en `docker.io/library/postgres` sans rien dire. Podman y voit un risque : un nom court peut désigner une autre image selon le registry interrogé (*typosquatting*, homonyme sur un registry interne). Il applique `registries.conf` : des **alias** connus (`alpine`, `nginx`, `postgres` ...) résolus sans question, et pour le reste la liste `unqualified-search-registries` — s'il y en a plusieurs, il vous demande de choisir. Sur Ubuntu cette liste ne contient que `docker.io`, donc les noms courts « marchent » ; sur Fedora ou une `podman machine`, ils déclenchent une question. Une image construite localement reçoit le préfixe `localhost/` : `podman build -t api:1.0` produit `localhost/api:1.0`. `podman images` affiche toujours le nom complet, sans magie.

PIÈGE

`latest` ne veut pas dire « la dernière version ». C'est un tag **par défaut** comme un autre, que le publieur choisit (ou pas) de déplacer ; il peut pointer vers une version vieille de deux ans. En production, `latest` est proscrit : déploiement non reproductible, retour arrière impossible.

2. Tag mouvant, digest immuable

Un **tag** est un pointeur : `postgres:16` désigne aujourd'hui `16.10`, demain `16.11`. Rien ne bouge sur votre disque, mais un nouveau `pull` ramènera autre chose. Un **digest** est l'empreinte SHA-256 du manifeste : `postgres@sha256:9d0d1f1e...`. Il est **calculé à partir du contenu**, donc :

- deux images de même digest sont bit à bit identiques, où qu'elles soient ;
- une image ne peut pas changer sans que son digest change ;
- on peut donc épingler un déploiement de façon absolue.

→ SÉCURITÉ

SHA-256 est une **fonction de hachage** : elle transforme n'importe quelle donnée en 64 caractères hexadécimaux, de façon déterministe (même entrée, même sortie) et à sens unique (impossible de fabriquer une entrée qui donne une sortie choisie). Changer un seul bit de l'image change entièrement l'empreinte. C'est le même principe qui identifie un *commit* Git : un identifiant qui *est* une preuve d'intégrité.

```
podman image inspect --format '{{.Digest}}' postgres:16-alpine
podman pull docker.io/library/postgres@sha256:9d0d1f1e... # parfaitement reproductible
```

Le compromis usuel en entreprise : un tag unique et immuable par build (`api:1.4.2` ou `api:2026.03.17-b318`), jamais réutilisé, et des outils de déploiement qui épinglent le digest.

3. Une image est un empilement de couches

Chaque instruction de construction qui modifie le système de fichiers produit une **couche** (*layer*) : un ensemble de fichiers ajoutés, modifiés ou supprimés par rapport à l'état précédent. L'image finale est la superposition de ces couches, plus un manifeste qui les liste et une configuration (commande par défaut, variables, utilisateur...).

	couche 4 : COPY app.jar	(60 Mo)
	couche 3 : le JRE	(180 Mo)
	couche 2 : paquets système	(30 Mo)
	couche 1 : base Debian slim	(75 Mo)

↑ lecture seule, partagées entre toutes les images qui les contiennent

→ LINUX

Le pilote de stockage `overlay` est un système de fichiers du noyau qui **superpose** des répertoires : des couches « basses » en lecture seule et une couche « haute » en écriture. Lire cherche du haut vers le bas ; écrire copie d'abord le fichier dans la couche haute (*copy-on-write*) ; supprimer crée un fichier fantôme (*whiteout*) qui masque sans retirer. Tout le comportement des images découle de ces trois règles.

Cette structure explique quatre comportements que vous constaterez sans cesse :

1. Le partage sur disque. Si vos douze microservices partent du même JRE, ces 180 Mo sont stockés **une seule fois**. La somme de la colonne `SIZE` de `podman images` dépasse donc largement l'espace occupé — `podman system df` donne le vrai chiffre.

2. Le transfert différentiel. Un `pull` ou un `push` ne transfère que les couches absentes : redéployer votre API ne transfère souvent que la couche du JAR.

3. L'immuabilité, y compris des erreurs. Si une couche ajoute un mot de passe et qu'une couche ultérieure le supprime, **le fichier est toujours dans l'image** : la couche suivante ne fait que le masquer, et quiconque a l'image peut le récupérer. Un secret ne doit jamais entrer dans un build (labo 08).

4. Le cache de construction. Les couches étant identifiées par leur contenu, le moteur réutilise celles qu'il a déjà (labo 04).

À RETENIR

Le partage des couches se fait à l'échelle de l'hôte ou du registry, pas de l'image : une couche identique dans deux images n'est stockée qu'une fois. En rootless, ce stockage est dans **votre** `home` (`~/.local/share/containers/storage`) : deux utilisateurs de la même machine ne partagent rien.

PIÈGE

Un tag désigne en réalité une **liste de manifestes**, un par architecture (`linux/amd64` , `linux/arm64` ...) ; le `pull` choisit celle de votre machine. Une image construite sur un MacBook Apple Silicon refuse donc de démarrer sur un serveur `amd64` : `exec format error` . - `-platform` force l'architecture.

4. Les commandes du quotidien

```
podman pull nginx:alpine           # télécharger sans lancer
podman images                      # lister les images locales
podman images --filter dangling=true # les images sans tag (couches orphelines)
podman history nginx:alpine        # les couches, leur taille et leur origine
podman image tree nginx:alpine     # les couches... en arbre, avec leur image d'origine
podman image inspect nginx:alpine  # métadonnées complètes en JSON
podman tag nginx:alpine mon-nginx:v1 # ajouter un nom (devient localhost/mon-nginx:v1)
podman rmi mon-nginx:v1            # supprimer un nom (et l'image si c'est le dernier)
podman system df                   # espace réellement occupé
```

Deux subtilités mal comprises :

`podman tag` **ne copie rien**. Il ajoute une étiquette sur la même image ; les deux noms désignent le même `IMAGE ID`. Symétriquement, `rmi` sur une image portant deux tags ne supprime que le tag : les données ne partent que quand le dernier nom disparaît.

Une image « dangling » (`<none>:<none>`) n'est pas un déchet mystérieux. C'est une image dont le tag a été déplacé vers une version plus récente : elle a perdu son nom mais occupe toujours le disque — le résidu normal des reconstructions successives.

Sortir une image du moteur

```
podman save -o api.tar mon-api:1.0 # archive au format docker-archive
podman save --format oci-archive -o api.tar mon-api:1.0 # même chose, au format OCI standard
podman load -i api.tar # réimporte l'image, tags compris
```

Utile quand la cible n'a pas accès au registry (site isolé). Ne pas confondre avec `export` / `import`, qui aplatissent le système de fichiers **d'un conteneur** et perdent couches et configuration (`CMD`, `ENV`, `EXPOSE` ...).

5. Les registries

Un registry est un service HTTP qui stocke des couches et des manifestes, derrière une API standardisée (`/v2/...`) que tous les outils parlent.

→ HTTP

Une API REST expose des *ressources* à des URL et les manipule avec les verbes HTTP : `GET /v2/_catalog` liste les dépôts, `HEAD /v2/api/manifests/1.0` renvoie le digest dans un en-tête, `PUT` pousse une couche. `curl` suffit donc pour interroger un registry — vous le ferez au labo pratique. C'est la mécanique d'une API Spring Boot.

Type	Exemples	Usage
Public	Docker Hub, <code>ghcr.io</code> , <code>quay.io</code>	Images de base et logiciels du commerce
Privé managé	AWS ECR, Azure ACR, Google AR	Images maison, hébergées chez le cloud
Privé auto-hébergé	Harbor, Nexus, GitLab Registry	Images maison, contrôle total, scan de vulnérabilités

Le cycle en entreprise :

```
podman login registry.masociete.be
podman tag api:1.4.2 registry.masociete.be/paiement/api:1.4.2
podman push registry.masociete.be/paiement/api:1.4.2
```

Trois points à connaître :

- **podman login stocke le jeton** dans `${XDG_RUNTIME_DIR}/containers/auth.json`, un fichier temporaire effacé à la déconnexion — là où Docker écrit dans `~/.docker/config.json`, en clair (base64) et pour toujours. Sur un agent de CI, on préfère des identifiants éphémères.
- **Podman exige TLS.** Un registry en HTTP simple — comme celui que vous lancerez sur `localhost:5000` — est refusé (`http: server gave HTTP response to HTTPS client`) tant que vous n'avez pas dit `--tls-verify=false` ou déclaré le registry `insecure = true` dans `registries.conf`. Docker fait une exception silencieuse pour `localhost` ; Podman non.
- **Docker Hub limite les téléchargements anonymes** (quota par IP) : sur une CI, cela donne des `toomanyrequests`, d'où l'usage d'un *pull-through cache* ou d'une copie interne des images de base.

6. En entreprise

Sur une stack Spring Boot + Angular :

- Les **images de base** (`eclipse-temurin`, `node`, `nginx`, `postgres`) sont recopiées dans le registry interne, souvent avec `skopeo copy` — l'outil frère de Podman qui copie d'un registry à l'autre sans rien télécharger. Personne ne tire d'Internet en production : quota, disponibilité, contrôle de ce qui entre.
- La CI construit `registry.interne/monapp/api:<version>` et `.../web:<version>`, puis pousse ; la version vient du tag Git ou du numéro de build. Un outil de **scan** (Trivy, Harbor, Grype) bloque les images porteuses de vulnérabilités critiques. Le déploiement référence une version précise, jamais `latest`.

À retenir

- Un nom complet est `registry/namespace/repository:tag` ; par défaut `docker.io`, `library` et `latest`. Podman affiche toujours ce nom complet et préfixe vos builds de `localhost/`.
- `latest` n'est pas « la plus récente » : c'est un tag par défaut, à bannir en production.
- Le **tag** peut bouger, le **digest** `sha256:...` identifie un contenu exact.
- Une image est un empilement de **couches en lecture seule**, partagées entre images, transférées de façon différentielle — et un fichier supprimé dans une couche ultérieure y reste : jamais de secret dans un build.
- `tag` ne duplique rien ; `rmi` retire d'abord un nom, pas des données. Podman exige TLS : `--tls-verify=false` seulement pour un registry local de test.
- `save` / `load` transportent une image complète ; `export` / `import` aplatissent un conteneur et perdent sa configuration.

Vocabulaire

repository : les versions d'une image. — **tag** : étiquette mouvante. — **digest** : empreinte SHA-256 immuable. — **manifest** : description des couches et de la configuration ; la **manifest list** en indexe plusieurs architectures. — **dangling image** : image ayant perdu son tag. — **layer** : couche de fichiers. — **overlay** : pilote qui superpose les couches. — **pull-through cache** : miroir local d'un registry public. — **image officielle** : namespace `library` sur Docker Hub. — **short name** : nom sans registry, résolu par `registries.conf`. — **skopeo** : copie et inspection d'images entre registries.